linux_audit

Security-Oriented Design and Development Report

A lightweight Linux audit daemon written in C

Features | Architecture | Security decisions | Findings | Roadmap

Summary

linux_audit is a small daemon that watches a Linux host and writes a readable log of process starts and stops, root login sessions and a system snapshot. It uses only libc, reads only from `/proc`, `passwd` and `utmp`, and never executes other programs or opens network sockets.

This report describes what the tool does, how it is built, and why specific design choices were made from a security point of view. It also lists the defects found during code review, how each was fixed, what was verified at runtime, the limits of the current design, and the improvements planned next.

Key figure	Value
Source size	6 modules, ~700 lines of C (sources and headers), zero third-party dependencies
Review result	9 findings (2 high, 4 medium, 2 low, 1 informational), all fixed
Build quality	0 warnings with -Wall -Wextra -Wpedantic; sanitizer build (ASan + UBSan) available
Memory model	Fixed-size static buffers; no dynamic allocation anywhere in the daemon

Report date: 30 October 2026

Scope: source review, remediation and runtime verification of the linux_audit codebase

1. Features

The daemon runs every 10 seconds after detaching from the terminal. Each capability below maps to one source module and has a specific security purpose.

Feature	Module	What it does	Security relevance
Process lifecycle logging	processes.c	Reads `/proc/<pid>/status` for each process, compares with the previous scan, logs PID, name, owner on start and PID, name on exit.	Detects unexpected programs, e.g. a new shell or miner appearing under an unusual user.
Root session tracking	sessions.c, users.c	Reads `utmp`, maps each user name to a UID via the passwd database and logs root sessions with TTY and source host. Logs only when the set changes.	Shows who has administrative access and from where; change-only logging keeps the log readable.
System snapshot	system_info.c	Records hostname, kernel, OS, architecture and memory usage when the daemon starts.	Gives each log a baseline for correlating events with the host state.
User database access	users.c	Loads users (bounded to 128), looks users up by UID, computes per-user process statistics.	Provides the name-to-UID mapping that makes root-session detection reliable.
Daemon lifecycle	daemon.c, main.c	Double fork, `setsid`, `umask 077`, descriptor cleanup, single-instance check, clean shutdown on SIGTERM/SIGINT.	Reduces exposure of inherited resources and avoids stale state or duplicate monitors.
Build targets	Makefile	`make` (production paths), `make dev` (/tmp paths, no root), `make debug` (ASan and UBSan).	Lets contributors test safely without root and catch memory errors early.

Example log output

```
=== Audit started [2026-10-30 09:21:40] ===
Hostname: vm
Kernel: 6.18.44-fc-v185 (#1 SMP PREEMPT_DYNAMIC @0)
OS Name: Linux
Architecture: x86_64
Memory: 3995 MB total, 5.45% used
----------------------------------------
[2026-10-30 09:21:50] Process started: PID 206 (sleep), UID 0 (root)
[2026-10-30 09:21:50] Process terminated: PID 200 (sleep)
Audit daemon stopped
```

The excerpt above is real output from a test run of the dev build. A root-session entry has the form `[time] Root session: user root, TTY pts/0, host 203.0.113.7` (illustrative address).

> **Positioning.** linux_audit is a lightweight monitor and a readable reference for Linux daemon and /proc programming. For tamper-resistant, kernel-level auditing in production, it should complement rather than replace the kernel audit framework (auditd) or eBPF-based tooling.

2. Architecture

The program is a single-process, single-threaded loop. Data flows from untrusted system sources, through narrow collection modules that validate and copy it into fixed structures, to the core loop that decides what to log.

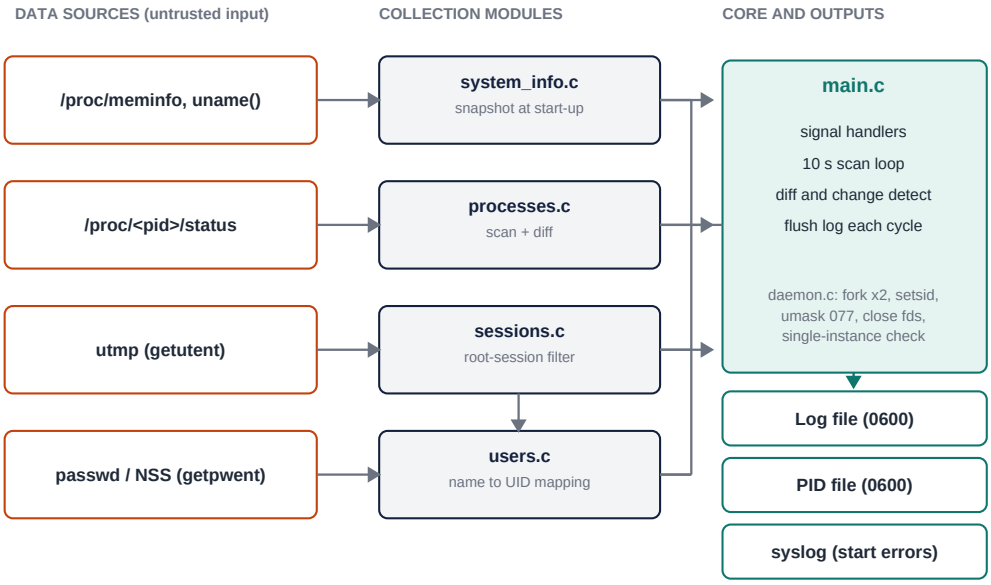

Figure 1 - Data flow. Orange borders mark inputs that must be treated as untrusted; teal borders mark outputs.

Module responsibilities

Module	Responsibility	Key interfaces
main.c	Orchestration: signal handlers, scan loop, change detection, ordered shutdown.	main(), handle_signal()
daemon.c	Detach from terminal, open log and PID file safely, refuse a second instance.	daemonize(), load_files(), remove_pid_file()
processes.c	Parse process status files, compare two scans, format events.	load_processes(), detect_new_processes()
sessions.c	Read utmp, resolve users, filter by UID, compare snapshots.	load_sessions(), get_sessions_by_uid(), sessions_equal()
users.c	Enumerate the passwd database, look up by UID, per-user counters.	load_users(), find_user_by_uid()
system_info.c	One-time host and memory snapshot.	get_system_info(), print_system_info()

One scan cycle

Step	Action
1	Read every numeric directory under /proc into the *current* process array (capped at 4096).
2	If this is not the first scan, compare *previous* and *current*; log each PID that appeared or vanished, then copy *current* into *previous*.
3	Read utmp, resolve user names to UIDs, keep UID 0 sessions.
4	If the root-session set changed, log the new state. Flush the log, sleep 10 seconds, and exit the loop once a termination signal has arrived.

3. Security Design Decisions

Each decision below states the risk it addresses and where it is implemented.

3.1 Minimal attack surface

- **No network, no child processes.** The daemon never opens a socket and never calls `system()`, `exec` or a shell, so there is no command-injection or remote path into it.
- **Read-only observation.** Apart from its own log and PID file it only reads system files.
- **No third-party code.** Only libc is used, so there is no dependency supply-chain exposure.

3.2 Memory safety by construction

- **Fixed-size static storage, no malloc.** Process, session and user arrays have compile-time limits (4096, 128, 128). This rules out leaks, use-after-free and double-free in the daemon.
- **Bounded copies everywhere.** All string copies use `snprintf` with the destination size; `strcpy` was removed. Fixed-width utmp fields are read with a precision limit because they are not guaranteed to end in a NUL byte.
- **Explicit limits instead of silent growth.** `load_users` stops at its capacity, and hitting the process cap is reported to syslog.
- **Zeroed records.** Every process and session record is cleared before it is filled, so values from an earlier scan cannot leak into a new record.
- **Sanitizer build.** `make debug` compiles with AddressSanitizer and UBSan for use during development.

3.3 Integrity of the audit data

An audit log is only useful if it is correct. A bug in the original code caused every logged-in user to be reported as **root**. The fix introduces an explicit sentinel so an unresolved user can never be confused with UID 0:

```
#define UNKNOWN_UID ((uid_t)-1)

if(!convert_name_to_uid(users, user_count, sessions[index].username, &sessions[index].uid)){
    sessions[index].uid = UNKNOWN_UID;   /* never defaults to root */
}
```

3.4 Safe file handling

```
int fd = open(LOG_PATH, O_WRONLY | O_APPEND | O_CREAT | O_NOFOLLOW | O_CLOEXEC, 0600);
```

- **O_NOFOLLOW** refuses to open a symlink at the final path component, so a local user cannot redirect a root-run daemon to overwrite another file.
- **Moved out of /tmp.** Production paths are `/var/log` and `/run`, which are not world-writable. The /tmp paths exist only in the dev build.
- **Restrictive permissions.** The log is created 0600 and `umask(077)` applies to every file the daemon creates, so audit data is not readable by ordinary users.
- **O_CLOEXEC and O_APPEND.** Descriptors are not inherited by any future child, and writes always append.

3.5 Process hygiene and lifecycle

- **Proper daemonization:** two forks and `setsid` so the process can never reacquire a controlling terminal; working directory set to `/`; all inherited descriptors closed; stdin/stdout/stderr bound to /dev/null.

- **Single instance.** Startup is refused if the PID file names a live process, preventing duplicate or conflicting monitors.
- **Graceful shutdown.** SIGTERM and SIGINT only set a flag (async-signal-safe); the main loop then flushes and closes the log and removes the PID file. Handlers are installed without SA_RESTART so `sleep` returns promptly.
- **Visible failures.** After detaching, stderr is unusable, so start-up errors go to syslog rather than disappearing.

3.6 Defensive parsing

- Every numeric field is parsed with `sscanf` into a typed variable and checked for success.
- A process that exits while its status file is being read is detected (empty name) and discarded instead of being logged as garbage.
- Names containing spaces (e.g. a browser helper process) are read completely, so a process cannot hide behind a truncated name.
- Timestamps use the reentrant `localtime_r` and a caller-supplied buffer rather than a shared static string.
- ANSI colour codes were removed from the log file so terminal escapes from the tool itself cannot mix with logged text.

4. Code Review Findings and Remediation

The original sources compiled but contained defects with security impact. All were reproduced or confirmed by inspection and fixed.

#	Finding	Severity	Impact	Fix
1	Log and PID files in /tmp with predictable names and no symlink protection.	High	A local user can redirect a root-run daemon to overwrite or corrupt files.	Moved to /var/log and /run; O_NOFOLLOW; mode 0600.
2	`endpwent()` never called: the second call to load_users returned 0 users (confirmed: 23, then 0). Unresolved users kept UID 0.	High	Audit data is wrong: any logged-in user is reported as a root session.	setpwent/endpwent around the loop; UNKNOWN_UID sentinel; records zeroed.
3	Off-by-one in load_users (`index > max`) wrote one element past the array; error return of -1 stored in size_t.	Medium	Out-of-bounds write on systems with more than 128 accounts.	Loop bound is index < max_users; function returns a bounded count.
4	`size_t count` compared with `< 0`, always false; a failed /proc read became SIZE_MAX and was passed to memcpy.	Medium	Crash (denial of service) when /proc is unavailable.	count is a signed int; error path logged and loop continues.
5	`log == NULL` tested instead of `*log == NULL`.	Medium	NULL dereference if the log cannot be opened.	Correct check, plus error reporting to syslog.
6	utmp fields (user, line, host) read with %s although not guaranteed NUL-terminated.	Medium	Over-read of adjacent memory into the log.	Precision-limited `%.*s` using field size.
7	`strcpy` of a 65-byte machine name into a 64-byte buffer.	Low	Latent one-byte overflow.	snprintf into correctly sized buffers.
8	Process names cut at the first space; Process struct not cleared between scans.	Low	Misleading names and stale fields in the log.	`%255[^\n]`; memset before parsing.
9	daemon.h used FILE without including stdio.h.	Info	Header not self-contained; fragile builds.	Header includes what it needs.

Quality improvements made alongside the fixes

- Process record reduced from 8,468 to 276 bytes by removing two never-populated fields (executable path and command line); static arrays shrink from about 66 MB to about 2 MB.
- Dead code removed (a fully commented-out file and a committed binary); unused fields in the system snapshot removed.
- Signedness and format-specifier warnings eliminated; unsigned IDs printed with %u.
- Root sessions logged only on change and with timestamps, instead of repeating every 10 seconds.

5. Verification

Verification combined static checks with runtime tests on a Linux host. The table separates what was actually tested from what was not.

Check	Method	Result
Compilation	gcc with -Wall -Wextra -Wpedantic	Passes with zero warnings (original code produced 10).
User enumeration	Test harness calling load_users twice and with a limit of 5	23, 23 and 5 users: repeatable and bounded.
Header hygiene	Compiling daemon.h on its own	Original failed; fixed version passes.
Daemon start-up	Ran dev build; checked log and PID file	Banner written, process start/stop events logged with timestamps.
Clean shutdown	Sent SIGTERM and SIGINT	Log shows the stop message; PID file removed; process exits.
Single instance	Started a second copy while one was running	Second copy refused; PID file unchanged.
Sanitizer build	Ran the ASan/UBSan binary	Started and stopped normally; the daemon closes stderr, so any sanitizer report would not be visible in that run.

Not yet verified: behaviour with real root sessions (the test host had no utmp entries), systems with more than 4096 processes, and sanitizer output captured to a file. These are included in the roadmap as test work.

6. Threat Model and Residual Risk

The table shows what the current design defends against and what it deliberately does not.

Threat	Mitigation in place	Residual risk
Local user plants a symlink at the log or PID path	O_NOFOLLOW, root-owned directories, mode 0600	Parent-directory permissions must be correct; only the last path component is protected.
Hostile or malformed data in /proc or utmp	Typed parsing, bounded copies, NUL-safe reads	Control characters inside names or hosts are written to the log unescaped (log injection).
Resource exhaustion by many processes	Fixed array limits, syslog warning at the cap	Above 4096 processes the list is truncated; truncation can cause false start/stop events and blind spots.
Attacker evades by short-lived processes or PID reuse	None (polling design)	Processes living less than 10 seconds, or a PID reused within one interval, are not seen.
Attacker with root stops or edits the daemon and its log	None	The log is not tamper-evident and is stored on the monitored host.

Threat	Mitigation in place	Residual risk
Log file grows until the disk is full	None yet	No rotation or size limit.
Race when two instances start at once	PID-file liveness check	Check and create are not atomic; a lock (flock) would close the window.
Forged or missing utmp records	None	utmp is a convenience source, not authoritative; it may be empty on systemd-only systems or containers.

7. Planned Improvements

The roadmap is ordered by security value. Items marked P1 address a residual risk from the previous section directly.

Pri.	Improvement	Why it matters	Approach
P1	Log sanitization	Prevents log injection and terminal escape abuse from hostile process names or hosts.	Escape non-printable bytes before writing; add a unit test with crafted names.
P1	Log rotation and size limits	Removes disk-fill denial of service.	Reopen log on SIGHUP for logrotate; optional built-in size cap.
P1	Atomic single-instance lock	Closes the start-up race.	Hold flock() on the PID file for the life of the process.
P1	Compiler and linker hardening	Raises the cost of exploiting any remaining memory bug.	Add -fstack-protector-strong, -D_FORTIFY_SOURCE=2, -fPIE -pie, -Wl,-z,relro,-z,now to the Makefile.
P2	Event-driven process monitoring	Catches short-lived processes that polling misses.	Use the netlink process connector (needs CAP_NET_ADMIN) with polling as fallback.
P2	PID-reuse detection	Stops a recycled PID looking like the same process.	Compare process start time from /proc/<pid>/stat together with the PID.
P2	Privilege reduction and service hardening	Limits damage if the daemon is ever compromised.	Systemd unit with NoNewPrivileges, ProtectSystem, restricted capabilities; seccomp filter.
P2	Authoritative login source	utmp may be absent or forged.	Read systemd-logind sessions or auth logs; track failed logins (btmp).
P3	Tamper-evident and remote logging	Defends the audit trail against a local attacker.	Hash-chain log entries; forward to a remote syslog or collector.
P3	Richer process context	Faster triage of suspicious processes.	Record executable path and command line (with truncation and sanitization).
P3	Scalable diffing	Removes the 4096 cap and O(n squared) cost.	Hash set of PIDs with dynamic capacity and an upper bound.
P3	Configuration and structured output	Easier deployment and machine analysis.	Config file for interval, paths and watched UIDs; optional JSON lines output; UTC timestamps.
P3	Automated testing and CI	Prevents regressions of the fixed bugs.	Unit tests for parsers, fuzzing of /proc and utmp parsing, sanitizer builds in CI.

Suggested sequence

Phase	Focus	Outcome
1	Log sanitization, rotation, flock, hardening flags	Closes every P1 residual risk with small, low-risk changes.

Phase	Focus	Outcome
2	Tests and CI, then PID-reuse detection	Locks in correctness before adding larger features.
3	Netlink events, logind, systemd unit	Moves from polling to near-real-time and reduces privileges.
4	Tamper-evident and remote logging	Makes the audit trail trustworthy even if the host is compromised.